جامعة الشارقة UNIVERSITY OF SHARJAH	**University of Sharjah** **College of Computing and Informatics** Department of Computer Science 1501366: Software Engineering Final Project, Fall 25-26 Due Date: Thursday, November 27th, 2025

Section 62

Group #4: SoftStruct

Phase 3: Test Specification Document

Group Members:

[redacted]

Shaikha Alfalasi – [redacted]

Table of Contents

Section 1……3

1.0 Introduction……3

1.1 Goals and Objectives……4

1.2 Statement of Scope……4

1.3 Major Constraints……5

Section 2……5

2.0 Test Plan……5

2.1 Software (SCIís) to be tested……5

2.2 Testing Strategy……6

2.2.1 Unit Testing……6

2.2.2 Integration Testing……6

2.2.3 Validation Testing……6

2.2.4 High-order Testing……7

2.3 Testing Resources and Staffing……7

2.4 Testing Work Products……8

2.5 Test Record Keeping……8

2.6 Test Metrics……8

2.7 Testing Tools and Environment……9

2.8 Test Schedule……10

Section 3……10

3.0 Test Procedure……10

3.1 Software (SCIís) to be tested……10

3.2 Testing Procedure……11

3.2.1 Unit Test Cases……11

3.2.1.2 Stubs and/or drivers for component i……11

3.2.1.3 Test cases component i……11

3.2.1.4 Purpose of tests for component i……11

3.2.1.5 Expected results for component i……11

3.2.2 Integration testing……24

3.2.2.1 Testing procedure for integration……24

3.2.2.2 Stubs and drivers required……24

3.2.2.3 Test cases and their purpose ……25

3.2.2.4 Expected results……26

3.2.2 Validation testing……26

3.2.3.1 Testing procedure for validation……26

3.2.3.3 Expected results……26

3.2.3.4 Pass/fail criterion for all validation tests……26

3.2.4 High-order testing (a.k.a. System Testing)……27

3.2.4.1 Recovery testing……27

3.2.4.2 Security testing……27

3.2.4.3 Stress testing……27

3.2.4.4 Performance testing……28

3.2.4.5 Alpha/beta testing……28

3.2.4.6 Pass/fail criterion for all validation tests……28

3.3 Testing resources and staffing……28

3.4 Test work products……29

3.5 Test record keeping and test log……29

TEST SPECIFICATION

1.0 Introduction

The Smart Community Fridge System provides users with a unified digital platform to create user accounts, locate community fridges, and access basic assistance features. This Test Specification Document defines the complete testing plan and test procedures for all implemented system functionalities. It ensures that user interactions, system behaviors, and integrated services operate correctly and meet the defined requirements.

The system includes three major use cases, each initiated by specific actors:

UC1: Select Role

Initiated by: Any user opening the application

This use case allows users to choose their interaction role within the system.

Sub-use cases:

UC1.1: Sign Up – New users (Donor, Beneficiary, Stakeholder) create an account.

UC1.2: Login – Existing users authenticate to access the system.

UC1.3: Provide Basic User Information – User details are securely stored in Firebase.

UC2: Locate Fridge to Receive Donation

Initiated by: Beneficiary

This use case enables beneficiaries to search for the nearest community fridge.

Sub-use case:

UC2.1: View Fridge Items

(Note: The physical act of receiving a donation is outside the system boundary.)

UC3: Locate Fridge to Donate

Initiated by: Donor

This use case enables donors to locate a nearby fridge for real-world donation.

(Note: The physical act of donating is outside the system boundary.)

An **AI Chat Assistant** is also available to all logged-in users for basic guidance.

1.1 Goals and objectives

The goals and objectives of the testing activities are to:

- Ensure the correctness and reliability of all implemented software functions.
- Validate that authentication, user role management, map integration, and AI chat operate as expected.
- Detect defects early and maintain system stability.
- Confirm that each use case behaves according to the system requirements.
- Assess the usability and performance of the user interface.

1.2 Statement of scope

The Smart Community Fridge System will be tested across multiple levels, beginning with unit testing (using white-box testing methods), followed by integration testing (using black-box techniques), validation testing against the defined use cases, and concluding with high-order system testing, including usability, performance, and public beta testing. A number of key system behaviors and design principles will be verified throughout the testing process.
The **role-selection interface** should be intuitive and easy for users to navigate.
The **sign-up and login workflows** must operate reliably, and all user information should read from and write to Firebase correctly.
The **fridge location functionality**, including map rendering and marker display, must be responsive and accurate.

The **AI chat assistant** must respond consistently, handle user input gracefully, and provide clear and helpful guidance.

All navigation flows within the application should operate smoothly, with no unexpected interruptions or crashes. Additionally, the system will be evaluated for overall usability, responsiveness, and compatibility across browsers and device types. Help content and on-screen guidance must be clear and sufficient for first-time users. The software should

load quickly, maintain stable performance, and handle typical user interaction without errors.

1.3 Major constraints

- Only the currently implemented software features are available for testing (the 3 use-cases)
- Firebase free-tier usage may introduce performance limitations for authentication and database operations.
- A stable internet connection is required for Firebase services and for loading Google Maps.
- The Google Maps integration displays fridge locations and can generate directions; however, navigation opens in the external Google Maps application instead of running inside our app. Full in-app navigation is not yet supported.
- Testing is limited to devices and browsers available to the development team (Chrome on desktop and Android/iOS mobile).

2.0 Test Plan

This section explains how the Smart Community Fridge System will be tested and what is needed to perform testing successfully. It outlines the main testing levels, the order in which they will be carried out, and the tools, resources, and responsibilities involved. The goal is to ensure that all implemented features work correctly and meet the system requirements.

2.1 Software (SCIs) to be tested

Interface Components to be Tested

- Role Selection Screen

- Sign Up & Login Screens

- Fridge Location (Map) Screen

- Home Screen/Dashboard

- AI Chat Interface

Application Logic Components to be Tested

- Authentication Logic

- User Role Handling
- Firebase Read/Write Functions
- Map Loading & Marker Display
- AI Chat Message Handling

Data Components to be Tested

- Firebase Authentication
- Firebase User Data Storage

External Service Components to be Tested

- Google Maps API Integration
- AI Chat Response Service

2.2 Testing strategy

The overall strategy for software testing is described.

2.2.1 Unit testing

Unit testing will focus on verifying individual components of the Smart Community Fridge System before they are combined with other modules. The main strategy is to test the smallest functional units (such as authentication logic, Firebase read/write operations, map initialization functions, and AI chat message handling) independently to ensure that each unit performs correctly. White-box testing methods (such as statement and branch coverage) will be used to check internal logic and detect defects early. Only components developed by the team will undergo unit testing.

2.2.2 Integration testing

Integration testing will verify that the different modules of the system work together correctly. Components will be integrated in a logical order, starting with backend services (Firebase Authentication and Database) and then connecting them to the frontend interfaces. After that, external services such as Google Maps and the AI chat engine will be added. The strategy combines a bottom-up approach for backend integration and a simplified Big-Bang approach for UI components due to the small size of the system. The goal is to ensure smooth data flow and proper communication between modules.

2.2.3 Validation testing

Validation testing will confirm that the implemented system meets the functional requirements of the defined use cases. Each use case (Select Role, Sign Up, Login, Locate Fridge to Receive Donation, Locate Fridge to Donate, and AI Chat interaction) will be tested from the user's perspective without considering internal code structure. Black-box techniques such as equivalence partitioning and boundary value analysis will be applied. Validation will be performed after unit and integration testing are completed, ensuring that the system behaves as the requirements specify.

2.2.4 High-order testing

High-order testing (system-level testing) will evaluate the overall behavior, performance, and usability of the full application. This includes usability testing for navigation and interface clarity, performance testing for map loading and Firebase response times, compatibility testing across devices and browsers, and basic security testing for login failures and restricted access. Alpha testing will be conducted internally by the development team, followed by informal beta testing with external users. The goal is to ensure that the system is stable, user-friendly, and reliable under realistic usage conditions.

2.3 Testing resources and staffing

The testing of the Smart Community Fridge System will be conducted using the resources available to the development team. The main testing resources include:

- **Devices:** Windows laptops using Google Chrome, and mobile devices running Android/iOS.
- **Software Tools:** Visual Studio Code (VS Code), Flutter development tools, Firebase Console, Google Maps API tools, Chrome Developer Tools, and the AI chat service integrated into the system.
- **Network Requirements:** A stable internet connection is required for Firebase authentication, database operations, AI chat communication, and Google Maps loading.

Testing activities will be carried out by the project team members:

- Test Team Leader - Shaikha Afalasi
- Unit Testing - All members
- Integration Testing - [redacted]
- Validation Testing - [redacted]
- High-Order Testing (System Testing) - [redacted]

Due to the small project scope, no independent testers are necessary. The team collectively shares responsibility for executing tests, reporting defects, and verifying corrections.

2.4 Test work products

Firebase Console Debug View
This tool will be used during testing to monitor authentication activity and database read/write operations. It assists in verifying that Firebase functions behave as expected.

AI Chat Test Log
A simple log will be maintained to record inputs sent to the AI assistant and the responses returned. This work product helps confirm that the AI behaves consistently and provides appropriate guidance.

Test Execution Log
A test log will be kept to document executed test cases, their outcomes, and any issues discovered. This log supports tracking testing progress and identifying components that require additional attention.

2.5 Test record keeping

Test records for the Smart Community Fridge System will be maintained using a simple and organized process. A **test log** will be used to document each test that is executed, including the test name, the component being tested, the expected outcome, the actual outcome, and a pass/fail result. This log will be maintained in a shared Google Sheet so that all team members can update it during testing.

An **error log** will also be maintained to record any issues or unexpected behaviors observed during unit, integration, validation, or system testing. Each error entry will include a brief description of the issue, the related component, steps to reproduce it, and its current status (open or resolved).

In addition, supporting evidence such as screenshots or console output from Visual Studio Code, Firebase Console, or Chrome Developer Tools will be saved alongside the test logs. These materials help verify that the system behaves as intended and assist in debugging when issues arise.

All logs and supporting files will be stored in the team's shared project folder, ensuring that the testing information is accessible, organized, and ready for inclusion in the final documentation.

2.6 Test metrics

The following metrics will be used to assess the completeness, quality, and effectiveness of the testing process applied to the Smart Community Fridge System.

Function Point Metric

This metric provides an estimate of the functional size of the implemented system and helps determine the amount of testing effort required for each feature based on its inputs, outputs, and interactions.

Cyclomatic Complexity

Cyclomatic Complexity is used to identify components that contain more complex decision logic, such as authentication workflows, Firebase read/write operations, and AI message handling. These modules will therefore receive more focused and extensive unit testing.

Breadth of Testing

Breadth of Testing measures how much of the system's functionality has been covered by the test cases. This ensures that all major screens, workflows, and features included in the current implementation are tested.

Depth of Testing

Depth of Testing evaluates how thoroughly each feature is tested by examining the percentage of independent logic paths and internal conditions covered by the test cases. This ensures that components are not only tested broadly but also tested in detail.

Fault Profiles

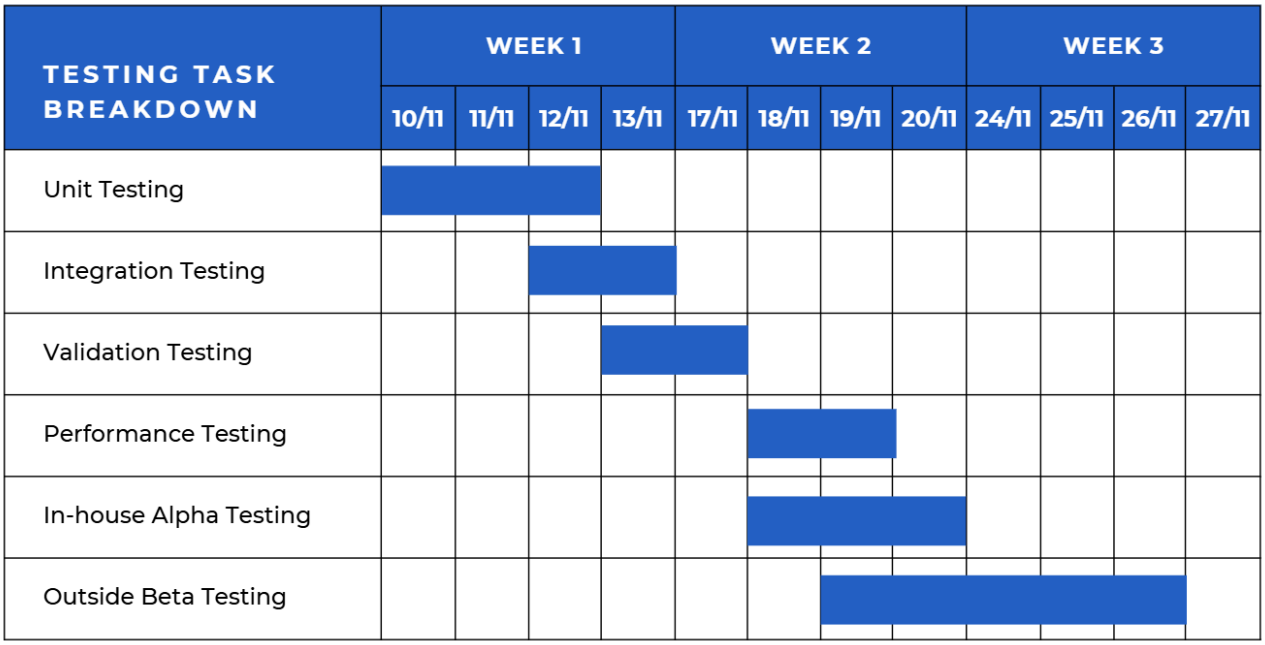
Fault Profiles categorize and track defects discovered during testing based on their type and severity. This helps the team prioritize critical issues and identify recurring problem areas that may require additional attention.

2.7 Testing tools and environment

Visual Studio Code (VS Code) and the Flutter development environment will be used as the primary testing tools and runtime environment for the Smart Community Fridge System. Testing will also rely on the Firebase Console to monitor authentication attempts, database read/write operations, and system behavior during execution.

Google Chrome will be used as the main browser for running and testing the application, supported by Chrome Developer Tools for inspecting logs, network activity, and UI behavior. The Google Maps API dashboard will assist in monitoring map-related requests made during testing.

Test data (such as sample user accounts and test inputs for the AI chat assistant) will be created and used for unit, integration, and system testing. No additional specialized



hardware or tools are required for testing the current implementation.

2.8 Test schedule

3.0 Test Procedure

This section describes a detailed test procedure including test tactics and test cases for the software.

3.1 Software (SCIs) to be tested

Interface Components to be Tested

- Role Selection Screen
- Sign Up & Login Screens
- Fridge Location (Map) Screen
- Home Screen/Dashboard
- AI Chat Interface

Application Logic Components to be Tested

- Authentication Logic
- User Role Handling
- Firebase Read/Write Functions
- Map Loading & Marker Display
- AI Chat Message Handling

Data Components to be Tested

- Firebase Authentication
- Firebase User Data Storage

External Service Components to be Tested

- Google Maps API Integration
- AI Chat Response Service

3.2 Testing procedure

The overall procedure for software testing is described.

3.2.1 Unit test cases

The procedure for unit testing is described for each software component (that will be unit tested) is presented. This section is repeated for all components.

Interface Components

3.2.1.1 Testing procedure for component: Role selection screen

This test ensures the Role Selection Screen loads correctly, shows all role options, and opens the correct screen when a role is chosen, without any crashes or unexpected behavior.

3.2.1.2 Stubs and/or drivers for component: Role selection screen

- None.

3.2.1.3 Test cases component: Role selection screen

- Screen Loads Correctly
  Verify that the Role Selection Screen appears without errors when accessed.
- Display of All Roles
  Check that all user roles are shown clearly on the screen.
- Role Selection Functionality
  Select each role and confirm that the correct screen opens for that role.

3.2.1.4 Purpose of tests for component: Role selection screen

The purpose of these tests is to ensure the Role Selection Screen loads correctly, shows all roles clearly, and opens the correct screen when a role is chosen, allowing users to start using the app without issues.

3.2.1.5 Expected results for component: Role selection screen

- The Role Selection Screen loads smoothly with no errors.
- All available roles are visible and clearly labeled.
- Choosing any role opens the correct next screen.
- No crashes or unexpected behavior occur during interaction.

3.2.2.1 Testing procedure for component: Sign Up & Login Screens

The testing procedure for the Sign Up & Login Screens involves verifying that both screens load correctly, accept user input, validate entered data, and communicate properly with Firebase Authentication.

3.2.2.2 Stubs and/or drivers for component: Sign Up & Login Screens

Firebase Authentication acts as the driver by processing all Sign Up and Login requests and returning the appropriate success or error messages, and no stubs are required.

3.2.2.3 Test cases component: Sign Up & Login Screens

- The Role Selection Screen loads correctly without errors.

- Verify Sign Up screen loads with all required fields (email, password, name, role)
- Test successful user registration with valid credentials
- Test login with correct email and password
- Selecting a role opens the correct next screen.
- Verify error messages for invalid email formats
- Confirm short passwords are rejected with appropriate warnings
- Test empty field validation
- Verify navigation to correct screen after successful authentication

3.2.2.4 Purpose of tests for component: Sign Up & Login Screens

These tests ensure that the Sign Up and Login screens display correctly, accept valid data, reject invalid input, communicate properly with Firebase Authentication, and navigate to the correct screen after a successful login or registration.

3.2.2.5 Expected results for component: Sign Up & Login Screens

- Both screens load smoothly with no UI errors.
- Valid user input is accepted and sent to Firebase successfully.
- Invalid emails, short passwords, or empty fields produce appropriate error messages.
- Successful login or registration redirects the user to the correct next screen.
- The app remains stable with no crashes or unexpected behavior.
- After successful Sign Up, user information (email, name, role, uid, timestamps, status) is created correctly in Firebase.

3.2.3.1 Testing procedure for component: Fridge Location (Map) Screen

The Fridge Location Screen will be tested using black-box methods by opening the screen, loading fridge locations, and selecting fridges to view their details. These tests verify that fridge markers load correctly and that selecting a fridge displays the proper information.

3.2.3.2 Stubs and/or drivers for component: Fridge Location (Map) Screen

For this component, we used a stub that provides a set of fixed fridge locations directly in the code, allowing us to display markers on the map without needing real data from the database. A driver was also used, which is a small test program that

opens the Map Screen, loads the Google Map, and checks that the markers appear correctly and can be tapped.

3.2.3.3 Test cases for component: Fridge Location (Map) Screen

- Verify that the Fridge Location Screen loads correctly and displays all fridge markers in their proper locations.
- Ensure that selecting a fridge marker opens its information sheet with the correct fridge details.
- Confirm that the displayed fridge content image loads successfully and matches the fridge's current status (empty, well-stocked, or full).

3.2.3.4 Purpose of tests for component: Fridge Location (Map) Screen

These tests ensure that the screen functions smoothly, displays accurate fridge information, and allows users to navigate to different fridge locations reliably.

3.2.3.5 Expected results for component: Fridge Location (Map) Screen

- The map loads successfully and displays all fridge markers accurately.
- Selecting a marker shows the correct fridge information.
- The fridge content image displays properly and matches the fridge's status.
- All user interactions on the screen operate smoothly without errors.

3.2.4.1 Testing procedure for component: Home Screen/Dashboard

The Home Screen will be tested using black-box methods by navigating to the screen, interacting with all UI elements, and verifying proper data loading and navigation. Tests will cover the bottom navigation, quick stats display, fridge lists, and AI chat integration.

3.2.4.2 Stubs and/or drivers for component: Home Screen/Dashboard

For this component, we used stubs that provide fake fridge data and user information so the Home Screen can display its content consistently. A driver was then used to open the Home Screen and check that it shows the correct options based on the user's role and that moving between screens works properly.

3.2.4.3 Test cases for component: Home Screen/Dashboard

- Home Screen loads successfully with the correct username and user role, and no errors appear during initialization.
- Bottom navigation buttons work properly and open the correct screens when tapped.
- Quick stats such as "Available Fridges", "Total Fridges", and "Meals Shared" display accurate values.
- All fridges appear in the list, and each fridge card shows the correct name, location, status, and supports the favorite icon functionality.
- The search bar is functional, allows filtering fridges by name or status, and clearing the search returns the full list.
- The AI chat button opens the chat interface correctly, shows the greeting message, provides usable suggestions, and closes without issues.
- A loading indicator appears while data is being fetched, an empty state is shown if no fridges are available, and errors are handled properly when data fails to load.
- Screen content adapts based on the user's role, and the displayed information matches the appropriate role requirements.

3.2.4.4 Purpose of tests for component: Home Screen/Dashboard

The purpose of these tests is to ensure the Home Screen serves as a reliable navigation hub, displays accurate and relevant information to users, handles all user interactions smoothly, and adapts content appropriately based on user roles and permissions.

3.2.4.5 Expected results for component: Home Screen/Dashboard

- Home Screen loads successfully with no errors.
- Correct username and user role are displayed.
- Navigation buttons open the correct screens.
- Quick stats show accurate values.
- Fridge list displays correct information and allows favorites.
- Search bar filters and restores the list properly.
- AI chat opens, responds, and closes correctly.
- Loading, empty, and error states display properly.

3.2.5.1 Testing procedure for component: AI chat interface

The interface will be tested by sending various questions, confirming messages are displayed, processed by the AI service, and returned answers appear correctly. Response time and answer relevance will also be evaluated.

3.2.5.2 Stubs and/or drivers for component: AI chat interface

A stub will be used to replace the real AI service during unit testing. The stub will return simple, fixed responses such as "Test reply" to ensure that the chat UI shows messages appropriately. Driver will be used for a chat message simulator for testing user input and AI response flow

3.2.5.3 Test cases component: AI chat interface

- A valid user inquiry will be sent, and the AI's response will be displayed correctly.
- Verify chat bubble styling (user vs AI)
- An empty input will be rejected, and no message will be sent.
- A long user question will be accepted, processed, and shown properly in the chat.
- If the AI service fails, an error message will appear instead of the app crashing.

3.2.5.4 Purpose of tests for component: AI chat interface

The purpose of these tests is to ensure the chat interface receives user messages correctly, displays AI responses accurately, handles errors safely, and performs efficiently during these interactions.

3.2.5.5 Expected results for component: AI chat interface

- Valid user inquiries are sent successfully, and the AI responses appear correctly.
- Empty inputs are blocked, and no message is sent.
- Long user questions are displayed properly without layout issues.
- If the AI service fails, an error message is shown and the app remains stable.
- The interface responds efficiently without noticeable delays.

Application Logic

3.2.6.1 Testing procedure for component: Authentication logic

This test checks that authentication works correctly by testing valid and invalid login attempts and confirming proper error handling.

3.2.6.2 Stubs and/or drivers for component: Authentication logic

For this component, Firebase Authentication is used directly without any stubs. A driver was implemented to simulate different login attempts and verify that the authentication logic works correctly, including handling valid and invalid credentials.

3.2.6.3 Test cases component: Authentication logic

- A valid username and password will be accepted.
- Invalid username or password will be rejected.
- Empty login fields will not be accepted.
- The system will ensure only registered users can access the app.

3.2.6.4 Purpose of tests for component: Authentication logic

The purpose of these tests is to ensure correct verification and access control.

3.2.6.5 Expected results for component: Authentication logic

- The system accepts valid usernames and passwords and grants access.
- Invalid credentials are rejected with an appropriate error message.
- Empty login fields trigger a prompt indicating that input is required.
- Only registered users are able to successfully log in.

3.2.7.1 Testing procedure for component: User Role Handling

This test validates that the system correctly stores, retrieves, and interprets the user's selected role (Donor or Beneficiary) during the authentication process and navigation.

3.2.7.2 Stubs and/or drivers for component: User Role Handling

The Firebase Database driver retrieves user role data after login. Stub will be used for mock user role data for testing different role scenarios

3.2.7.3 Test cases component: User Role Handling

- Verify that the user's selected role is correctly saved in Firebase after Sign Up.
- Ensure each role opens the correct main page (Donor → Donor page, Beneficiary → Beneficiary page).
- Confirm that the correct role label appears wherever it is displayed in the UI.

3.2.7.4 Purpose of tests for component: User Role Handling

These tests ensure that user roles are correctly assigned to new accounts, accurately retrieved after login, and properly used to control navigation within the app. They also verify that no incorrect screens are opened for the wrong role and that the system handles missing or invalid role data safely.

3.2.7.5 Expected results for component: User Role Handling

- User role is always stored correctly upon registration.
- Login retrieves the correct role from Firebase.
- Donors always open the Donor main page; Beneficiaries always open the Beneficiary main page.
- No crashes occur when invalid role data is retrieved.

3.2.8.1 Testing procedure for component: Firebase Read/Write Functions

This testing procedure checks that reading from and writing to Firebase works correctly. It confirms that data is saved in the right place, can be read back properly, and updates when needed. Simple test inputs are used to make sure the functions behave well during normal use and when something unusual happens.

3.2.8.2 Stubs and/or drivers for component: Firebase Read/Write Functions

Stubs are used to supply fixed test data that helps check how the component reads and writes information. The drivers use simple Firebase read and write calls to activate the component and confirm that it behaves correctly during testing.

3.2.8.3 Test cases for component: Firebase Read/Write Functions

- Write operations save data to the correct location.

- Invalid inputs show the proper warning messages.
- Read operations return the correct stored data.
- Updates change only the intended fields and leave everything else the same.
- Deleted entries no longer appear in any results.
- The system handles missing data or connection issues properly.
- Add/remove fridge favorites.
- Update fridge status.

3.2.8.4 Purpose of tests for component: Firebase Read/Write Functions

These checks keep the link between the app and Firebase working right. When saving or grabbing data, they make sure things run smoothly even if something goes wrong. Thanks to solid verification, every part using live info acts steady without risks.

3.2.8.5 Expected results for component: Firebase Read/Write Functions

- All write operations store data in the correct place.
- Reading returns accurate and up-to-date information.
- Updates change only the intended fields.
- Deleted entries no longer appear anywhere in the database.
- Errors are handled with clear and correct responses.
- All Firebase operations run smoothly and consistently.

3.2.9.1 Testing procedure for component: Map Loading & Marker Display

The Map Loading component will be tested in a white box fashion. Test data with predefined fridge locations will verify proper map initialization and marker rendering.

3.2.9.2 Stubs and/or drivers for component: Map Loading & Marker Display

A stub is used to provide a predefined fridge location data during testing, while the actual Google Map loads normally in the app. A simple driver script is used to trigger marker interactions to verify that markers appear and respond correctly.

3.2.9.3 Test cases for component: Map Loading & Marker Display

- Map loads correctly with Sharjah as the default starting location and the appropriate zoom level.
- All predefined fridge locations appear as map markers.
- Each marker shows the correct fridge name and status.
- Clicking a marker sends the correct fridge ID through the JavaScript channel, and the app receives the event properly.

3.2.9.4 Purpose of tests for component: Map Loading & Marker Display

The purpose of these tests is to ensure that Google Maps loads correctly, all fridge location markers appear in the right places, marker colors reflect each fridge's status, and JavaScript communication between the map and Flutter works reliably.

3.2.9.5 Expected results for component: Map Loading & Marker Display

- Display all 8 fridge markers at correct geographic coordinates
- Apply appropriate colors to markers based on fridge status
- Handle marker click events and communicate with Flutter app
- Automatically adjust map view to show all markers

3.2.10.1 Testing procedure for component: AI Chat Message Handling

This test checks that the message-handling logic processes user messages, formats them correctly for the AI service, and retrieves in-app data when requested. It also verifies that AI responses and errors are handled safely.

3.2.10.2 Stubs and/or drivers for component: AI Chat Message Handling

A stub will simulate AI responses and also simulate in-app data retrieval results, allowing the message-handling logic to be tested independently.

No driver is required.

3.2.10.3 Test cases component: AI Chat Message Handling

- Valid messages are processed and sent to the AI service or internal data source in the correct format.
- Queries for in-app information (such as number of available fridges) return the correct data.
- If the AI service returns unexpected or malformed data, the component handles it safely.
- When the AI service is slow or unreachable, the component reports an error without affecting the system.

3.2.10.4 Purpose of tests for component: AI Chat Message Handling

The purpose of these tests is to ensure the message-handling logic processes user queries correctly, retrieves accurate in-app data when requested, and manages AI responses or failures without disrupting the system.

3.2.10.5 Expected results for component: AI Chat Message Handling

- Valid messages are formatted properly and sent to the correct destination (AI service or in-app data source).
- Queries for app data return accurate, up-to-date information.
- Unexpected AI responses are handled without errors.
- AI delays or failures result in clear error handling while the system remains stable.

Data Components

3.2.11.1 Testing procedure for component: Firebase Authentication

Authentication features such as sign-up, login, logout, and password reset will be tested using a variety of valid and invalid inputs to verify correct functionality and proper error handling.

3.2.11.2 Stubs and/or drivers for component: Firebase Authentication

The Firebase Authentication SDK will act as the driver during testing. It processes all authentication requests directly through the Firebase backend, so no stubs are required.

3.2.11.3 Test cases for component: Firebase Authentication

- Sign up with valid email and password
- Sign up with an already registered email
- Password reset functionality
- Log in with correct credentials
- Log in with an incorrect password
- Log out successfully

3.2.11.4 Purpose of tests for component: Firebase Authentication

These tests ensure that the component correctly authenticates users, blocks invalid attempts, and manages login and logout operations reliably.

3.2.11.5 Expected results for component: Firebase Authentication

- New accounts are created successfully when valid information is provided.
- Login attempts with correct credentials are accepted.
- Incorrect or invalid login attempts are rejected with proper error messages.
- Users are logged out successfully when the logout operation is triggered.

3.2.12.1 Testing procedure for component: Firebase User Data Storage

This testing procedure ensures that user information such as name, email, password, user ID, role, and join date is stored correctly and retrieved when needed. It checks that user records are created properly, saved without errors, loaded correctly during sign-in, and updated smoothly when any user details change.

3.2.12.2 Stubs and/or drivers for component: Firebase User Data Storage

Stubs are used to supply sample user data in the same structure used by Firebase, allowing the component to be tested with consistent inputs. The drivers rely on the real Firebase Auth and Firestore operations to load, save, and update this data, ensuring the component works correctly during testing.

3.2.12.3 Test cases for component: Firebase User Data Storage

- A new user profile is created correctly after sign-up.
- Stored user information is retrieved accurately after login.
- Updating user details changes only the intended fields.
- New profile information appears immediately in the app.
- Unauthorized access or edits are blocked by Firebase security rules.
- User profile data stays consistent across all devices.

3.2.12.4 Purpose of tests for component: Firebase User Data Storage

These checks ensure that user information stays protected, loads correctly when needed, and updates smoothly without errors. Proper validation keeps private details secure, maintains data consistency, and helps the app operate reliably especially when handling custom preferences or login based features.

3.2.12.5 Expected results for component: Firebase User Data Storage

- New user profiles are created correctly in the database.
- User information loads immediately after login.
- Profile updates are saved properly and appear across the app.
- Unauthorized access is blocked by Firebase security rules.
- User data stays consistent across all devices and sessions.
- All storage operations run smoothly and reliably.

External Service

3.2.13.1 Testing procedure for component: Google Maps API Integration

The Google Maps widget will be tested by loading the map, displaying markers, and retrieving the user's location. Various map interactions (like zooming, panning, and marker selection) will be performed to ensure proper behavior.

3.2.13.2 Stubs and/or drivers for component: Google Maps API Integration

The Google Maps Flutter plugin acts as the driver during testing. It handles all map rendering and API communication, so no additional stubs are required.

3.2.13.3 Test cases for component: Google Maps API Integration

- Load the map successfully on the screen
- Display all fridge markers in the correct positions
- Tap a marker to view its information
- Retrieve and show the user's current location
- Drag the map to move to different areas

3.2.13.4 Purpose of tests for component: Google Maps API Integration

These tests ensure that the map displays correctly, markers appear in the right locations, and user interactions such as dragging and selecting markers work smoothly.

3.2.13.5 Expected results for component: Google Maps API Integration

- The map loads without issues
- All markers appear in accurate locations
- Marker taps show the correct fridge information
- The user's location is retrieved and shown correctly
- Dragging the map works smoothly and without interruptions

3.2.14.1 Testing procedure for component: AI Chat Response Service

The AI Chat Response Service will be tested by submitting different types of queries and verifying that the service retrieves the correct data and generates accurate responses. The procedure checks how the service processes location-based queries and returns structured information for the chat interface.

3.2.14.2 Stubs and/or drivers for component: AI Chat Response Service

A stub will be used during testing to simulate the AI's reply format while still relying on real data from the test database.
No driver is required.

3.2.14.3 Test cases component: AI Chat Response Service

- A query is made to find all fridges located in "Sharjah".
- A query is made to find the closest fridge to a user's current coordinates.

3.2.14.4 Purpose of tests for component:AI Chat Response Service

The purpose of these tests is to ensure the service processes queries correctly and returns accurate, complete, and relevant fridge location information.

3.2.14.5 Expected results for component: AI Chat Response Service

- The service returns the correct list of all fridges in Sharjah.
- The service returns the correct fridge that is closest to the user's coordinates.

3.2.2 Integration testing

The integration testing procedure is specified.

3.2.2.1 Testing procedure for integration

Integration testing verifies that the individual modules of the Smart Community Fridge System operate correctly when combined. The system will be integrated incrementally to minimize the number of bugs introduced at each stage. Integration will follow the logical flow of the following components:

1. Role Selection Module
2. Authentication Module (Sign Up / Login)
3. Role-Based Navigation Module
4. Doner Main Page (Map + Favourite Fridge)
5. Beneficiary Main Page (Map + Favourite Fridge)
6. Google Maps Integration
7. AI Chat Interface and Response Handling

Integration follows a bottom-up approach for backend services (Firebase Authentication + Database) and a Big-Bang UI integration for the Flutter screens.

After each module is integrated, the system will be tested using black-box testing to verify that data flows correctly and navigation works as expected.

3.2.2.2 Stubs and drivers required

Stubs

- **Fridge Data Stub:** Provides predefined fridge details (ID, name, location, status) so the map and favorites features can be tested without loading real data.

Drivers

- **Firebase Authentication Driver:** Performs login and signup attempts to verify that real Firebase authentication works correctly and user roles are stored properly.

- **AI Chat Driver:** Sends messages to the real open-source AI service to test how responses are received and handled by the chat interface.
- **Map Screen Driver:** Opens the map, loads fridge markers, and simulates marker interactions to confirm correct map behavior.

3.2.2.3 Test cases and their purpose

1. Role Selection → Authentication → Role Routing

- After selecting a role, the user must reach the Sign Up / Login screen.
- After logging in, the user must be directed to the correct main page based on the stored role.

2. Authentication → Firebase Database

- User data is stored correctly in Firebase after Sign Up.
- Login retrieves correct user information and role.
- Invalid accounts should not pass through integration.

3. Donor/Beneficiary Main Pages → Fridge Data

- Favourite fridges display correctly after retrieving data.
- The fridge list contains correct status and distance.

4. Map Page → Google Maps Integration

- Tapping a fridge marker navigates to Google Maps with the correct coordinates.
- The system should not crash when Google Maps is unavailable.

5. Chat Interface → AI Chat Response Service

- User messages are sent correctly from UI to handler.
- Chat response appears in the interface.
- Errors from the AI service are handled safely.

These tests confirm that modules interact correctly, forming a smooth, working system.

3.2.2.4 Expected results

The integrated system is expected to behave as follows:

- Correct navigation from role selection → authentication → main pages.
- All Firebase operations respond without failures.
- Map and favourite fridges appear correctly for both users.
- The map must display all available fridges with proper status and markers.
- AI chat interface exchanges messages without crashes.
- No major functionality breaks when modules are combined.
- The system remains stable during navigation or repeated interactions.

3.2.3 Validation testing

The validation testing procedure is specified.

3.2.3.1 Testing procedure for validation

The features and functions of the final system will be checked against the Software Requirements Specification to confirm that the system meets all stated requirements.

3.2.3.2 Test cases and their purpose

Features that correspond to the design requirements will be tested. These tests are used to verify that the implemented system performs the actions and the responses expected in the requirements.

3.2.3.3 Expected results

The system should work within the limits and behavior defined in the Software Requirements Specification. All screens and system responses should match the required functionality.

3.2.3.4 Pass/fail criterion for all validation tests

A feature passes validation if its behavior follows the intention of the requirement, even if it is not identical to the wording in the Software Requirements Specification.

A feature fails if it does not perform according to the requirement's expected behavior.

3.2.4 High-order testing (a.k.a. System Testing)

High-order testing checks the Smart Community Fridge System as a whole. Each test type briefly outlines the procedure, purpose, and how success is measured.

3.2.4.1 Recovery testing

Procedure: Simulate app interruptions such as sudden app closure, network loss, or switching between screens while data is loading.
Test Cases: App resumes without losing the map state, fridge data, or user login session.
Purpose: Ensure the system can recover gracefully from unexpected events.
Special Requirements: Stable Firebase connection and proper state-saving logic.
Pass/Fail Criteria: The app restores the last stable state without crashes or data loss.

3.2.4.2 Security testing

Procedure: Check Firebase security rules, user authentication behavior, and access control for all data paths.
Test Cases: Only registered users access the app.
Purpose: Ensure user information is protected.
Special Requirements: Firebase Auth and Firestore security rule verification.
Pass/Fail Criteria: Unauthorized actions are fully denied, and authorized ones succeed.

3.2.4.3 Stress testing

Procedure: Increase the number of map interactions, marker taps, screen changes, and repeated Firebase calls.

Test Cases: Opening the map many times, tapping multiple markers quickly, loading many fridges at once.

Purpose: Confirm the system stays stable under heavy usage.

Special Requirements: Test environment with repeated simulated actions.

Pass/Fail Criteria: No slowdown, freezing, or crashes during heavy activity.

3.2.4.4 Performance testing

Procedure: Measure how fast map data loads, how quickly markers appear, and how long Firebase read/write tasks take.

Test Cases: Time to load fridge markers, time to update a fridge status, login speed.

Purpose: Ensure the system responds quickly and smoothly.

Special Requirements: Stopwatch/timer measurements.

Pass/Fail Criteria: All actions complete within acceptable time (for example, map loads within 2-3 seconds).

3.2.4.5 Alpha/beta testing

Alpha: Testing by the development team using the internal build.

Beta: Testing by selected users such as classmates.

Test Cases: Full app usage (the 3 use-cases): sign up, login, map navigation, marker details, and general app flow.

Purpose: Gather feedback, find hidden issues, and improve usability.

Special Requirements: Controlled test group and feedback forms.

Pass/Fail Criteria: No major issues found, and user feedback confirms acceptable usability.

3.2.4.6 Pass/fail criterion for all validation tests

The system passes validation if all major functions work correctly, no critical errors appear, security rules hold, performance meets expectations, and real users can use the app without confusion. Any failure in core functionality, data accuracy, or security results in a failed test.

3.3 Testing resources and staffing

The testing of the Smart Community Fridge System will be conducted using the resources available to the development team. The main testing resources include:

- **Devices:** Windows laptops using Google Chrome, and mobile devices running Android/iOS.

- **Software Tools:** Visual Studio Code (VS Code), Flutter development tools, Firebase Console, Google Maps API tools, Chrome Developer Tools, and the AI chat service integrated into the system.
- **Network Requirements:** A stable internet connection is required for Firebase authentication, database operations, AI chat communication, and Google Maps loading.

Testing activities will be carried out by the project team members:

- Test Team Leader - Shaikha Afalasi
- Unit Testing - All members
- Integration Testing - Hour Omar
- Validation Testing - Mera Almekhlafi
- High-Order Testing (System Testing) - Qamariya Albadi

Due to the small project scope, no independent testers are necessary. The team collectively shares responsibility for executing tests, reporting defects, and verifying corrections.

3.4 Test work products

Firebase Console Debug View

This tool will be used during testing to monitor authentication activity and database read/write operations. It assists in verifying that Firebase functions behave as expected.

AI Chat Test Log

A simple log will be maintained to record inputs sent to the AI assistant and the responses returned. This work product helps confirm that the AI behaves consistently and provides appropriate guidance.

Test Execution Log

A test log will be kept to document executed test cases, their outcomes, and any issues discovered. This log supports tracking testing progress and identifying

components that require additional attention.

3.5 Test record keeping and test log

Test records for the Smart Community Fridge System will be maintained using a simple and organized process. A test log will be used to document each test that is executed, including the test name, the component being tested, the expected outcome, the actual outcome, and a pass/fail result. This log will be maintained in a shared Google Sheet so that all team members can update it during testing.

The report format for beta testers is below:

Beta Tester Report Form

Name _____

Defect Report	**Enhancement Report**
Was the defect found in the interface or in the application logic?	Will the enhancement be for the interface or application logic?
Where exactly did the defect occur (Sign Up, Login, Map Screen, Marker Tap, Profile Page, etc..)?	Which screen or module should the enhancement be added to?
Is the defect reproducible? If yes, how?	Is the enhancement cosmetic (visual) or functional (adds a new feature)?
Did the defect cause the app to freeze, slow down, or crash?	Describe the enhancement you would like to see:
Additional comments:	Additional comments:

Test Log

The following is an example of the test log format:

Date	Test Type	System	Module	Coordinator	Results
11-10	Unit Testing	Smart Community Fridge System	Authentication	Mariam	Feature working correctly.
11-11	Unit Testing	Smart Community Fridge System	Login Validation	Hour	Minor delays but functions properly.
11-12	Unit Testing	Smart Community Fridge System	Map Loading	Qamariya	Displays expected output.
11-13	Unit Testing	Smart Community Fridge System	Marker Display	Mariam	All markers load as intended.
11-14	Unit Testing	Smart Community Fridge System	Home Screen	Mariam	Displays the user profile, quick stats, and nearby fridges.
11-14	Unit Testing	Smart Community Fridge System	Fridge Status Colors	Noorah	Displays the correct color for the corresponding fridge.
11-15	Unit Testing	Smart Community Fridge System	User Data Storage	Mera	Users' information is stored correctly in the database.

11-16	Unit Testing	Smart Community Fridge System	Profile Update Logic	Reem	Verified with no issues.
11-17	Unit Testing	Smart Community Fridge System	AI Chat Response	Shaikha	Outputs correct response, but doesn't scroll automatically when AI responds.
11-18	Unit Testing	Smart Community Fridge System	Map-Flutter Bridge	Hour	Stable and smooth. Google Maps integrates well with Flutter.
11-19	Unit Testing	Smart Community Fridge System	Realtime Fridge Updates	Qamariya	Loads quickly with no errors and shows fridge contents in real-time.
11-20	Unit Testing	Smart Community Fridge System	Role Selection Screen	Shaikha	Displays the correct screen for every role.
11-21	Unit Testing	Smart Community Fridge System	Volunteer Map Flow	Noorah	Map works flawlessly and is easy to use.
11-22	Unit Testing	Smart Community Fridge System	Firebase Read/Write	Mera	Accurate and consistent.
11-24	Unit Testing	Smart Community Fridge System	Fridge Info Test	Reem	Works correctly. Info windows display the right fridge details.